CYBERBULLYING DETECTION SYSTEM

Ulaş Toprak

[1]*Software Engineering Department, Istanbul Atlas University, (220504037)*

Abstract

In this project, an automated system was developed to classify social media tweets into two categories: cyberbullying and non-cyberbullying (clean). The primary goal of this study is to demonstrate the mathematical inner workings of machine learning algorithms from scratch. Therefore, instead of using standard high-level libraries (such as Scikit-Learn, PyTorch, or TensorFlow), all algorithms and preprocessing steps were built entirely using raw NumPy matrix commands. To handle the severe class imbalance in the dataset—where cyberbullying tweets outnumber clean tweets—a dynamic class weighting mechanism was integrated directly into the gradient updates. Three different architectures were implemented and compared: Logistic Regression, Support Vector Machine (SVM), and a 1D Convolutional Neural Network (1D CNN). Benchmarked over a dataset of 47,692 tweets, the custom SVM model achieved a peak recall of 97.11%, proving to be the most reliable model for catching toxic content. On the other hand, by analyzing localized word sequences (Trigrams), the 1D CNN model reached the highest precision of 93.16%, making it the most accurate model with the lowest false alarm rate.

Keywords: *Machine Learning, NumPy, CNN, Support Vector Machines, Linear Regression Cyberbullying Detection*

1. INTRODUCTION

The immense volume of daily social media interactions makes the manual moderation of cyberbullying content virtually impossible, necessitating the deployment of automated detection systems. While contemporary high-level libraries abstract the underlying computational steps behind single-line implementations, they routinely obscure the core mathematical formulations, gradient flows, and optimization loops. The primary objective of this study is to examine and validate the internal formulations, architectural behaviors, and gradient metrics of classification algorithms using fundamental matrix operations within a cyberbullying detection pipeline. During the preprocessing stage, text inputs were stripped of punctuation, user mentions, and stop words, then successfully vectorized using custom TF-IDF matrices and padded token sequences. To counteract the severe class imbalance where toxic entries vastly outnumber clean responses, dynamic cost weights were injected directly into the mini-batch iterations to prevent the models from biasing toward the majority prior. Concurrently, three distinct algorithmic frameworks were evaluated and benchmarked: Statistical Regularized Logistic Regression, Margin-Maximizing Support Vector Machines driven by conditional sub-gradients, and context-aware 1D Convolutional Neural Networks optimized via sequential trigram configurations. Furthermore, this comparative benchmark highlights how distinct geometric and probabilistic boundaries adapt to noisy, non-standard text data under structural constraint penalties. The empirical evaluation heavily prioritizes the balance between precision and recalls over raw accuracy to ensure optimal detection coverage with minimal false alarms.

2. DATA PREPROCESSING AND EXPLORATORY DATA ANALYSIS

An initial exploration of the dataset containing 47,692 social media entries was conducted to understand its structural composition and to clean the textual inputs before model training.

2.1. DATASET DISTRIBUTION AND IMBALANCE COMPENSATION

The raw dataset originally contained multi-class annotations detailing specific sub-types of cyberbullying, including ethnicity, religion, age, gender, and other descriptive forms of online harassment, alongside non-cyberbullying records. To establish structured and focused boundary constraints for our evaluation frameworks, these granular partitions were mapped into a unified binary classification task:

- Class 0 (Clean / Non-Cyberbullying): 7,945 samples
- Class 1 (Cyberbullying): 39,747 samples

This explicit reduction revealed a severe structural asymmetry within the data partitions.

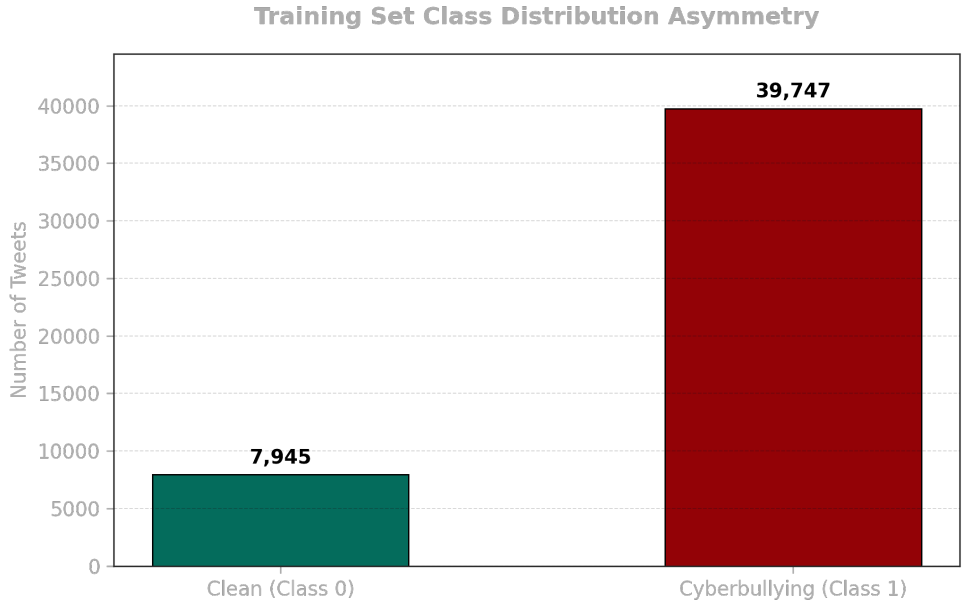


Figure 1 : Class distribution showing the severe structural imbalance between clean (Class 0) and cyberbullying (Class 1) samples.

To prevent the optimization algorithms from inherently biasing their decision thresholds toward the majority class, an analytical inverse class frequency weighting mechanism was integrated directly into the training loop:

$$weight(c) = \frac{N}{2 \times N_c}$$

Where N represents the total number of operational samples in the training split and N_C represents the specific count of class c. This equation dynamically yields a cost multiplier of 3.00 for Class 0 and 0.60 for Class 1, penalizing minority-class misclassifications more heavily during gradient updates.

2.2. TEXT NORMALIZATION AND CLEANSING PIPELINE

Raw social media text contains significant noise, such as hypermedia links and platform-specific syntax, which distorts feature weights. The textual inputs were normalized through a sequence of programmatic rules:

1. **Noise Stripping:** Regular expressions were deployed to completely remove user mentions (@user), hyperlinks (http/https), numerical digits, and standard punctuation.

2. **Case Unification:** All characters were mapped to lowercase to prevent the vocabulary generator from treating identical words as distinct entities due to capitalization differences.

3. **Character De-duplication:** Exaggerated, repetitive character variations typical of informal digital communication were parsed and restricted to a maximum of two consecutive identical letters using a custom lookup mechanism.

4. **Stopwords Elimination:** Grammatical function words that contain high document frequencies but lack semantic classification value (e.g., "the", "and", "is") were isolated and excluded from the token arrays.

3. FEATURE ENGINEERING AND VECTORIZATION

To transform processed textual tokens into numerical formats compatible with optimization routines, two distinct matrix-based vectorization pipelines were engineered.

3.1. VOCABULARY CONSTRUCTION AND LEAKAGE PREVENTION

Before generating mathematical representations, a global dictionary was constructed exclusively from the training partition (train_df). Restricting vocabulary definitions to the training split guarantees that the test set remains completely unseen, eradicating data leakage constraints.

- **Frequency Thresholding:** Unigrams and structural tokens appearing only once across the global document space were systematically pruned from the model's vocabulary using an explicit frequency constraint of (min_freq=2). This compression boundary plays a vital role in isolation mechanics by filtering out rare spelling anomalies, user typos, and highly specific lexical noise that would otherwise distort the vector space. From a structural engineering perspective, this threshold prevents the expansion of the sparse feature matrix, substantially lowering total vocabulary dimensionality, mitigating the risk of overfitting on anecdotal tokens, and optimizing down-stream matrix multiplication steps.

- **Padding Token Allocation:** Index 0 was explicitly reserved for the sequential padding indicator (<PAD>). This structured mapping ensures that placeholder elements do not distort semantic weights during downstream sequence processing. By defining index 0 as a neutral baseline, downstream layers can programmatically mask these boundaries, ensuring that only true linguistic signals contribute to the gradient computation.

3.2. MANUAL TF-IDF VECTORIZATION

For Logistic Regression and Support Vector Machines, text tokens were mapped into a continuous statistical coordinate space. The vectorizer computes the relative importance of words by multiplying Term Frequency () and Inverse Document Frequency ():

$$TF(t,d) = \frac{Frequency\ of\ token\ t\ in\ document\ d}{Total\ tokens\ in\ document\ d}$$

$$IDF(t) = \ln\left(\frac{1+N}{1+DF(t)}\right) + 1$$

Where N represents the absolute document count across the corpus and $DF(t)$ represents the number of documents containing token t.

- **Memory Optimization:** All compiled feature cells were explicitly cast as 32-bit floating-point structures (np.float32). This architectural decision cuts the memory footprint by 50% compared to standard 64-bit defaults, significantly accelerating matrix inner-product evaluations during training loops.

3.3. TOKEN ENCODING AND SEQUENCE PADDING (FOR CNN)

While linear models evaluate documents via bag-of-words arrays where sequence layout is dissolved, deep architectures require the spatial proximity of tokens to learn temporal context.

- **Integer Index Mapping:** Raw text tokens were translated into chronological integer vectors based on their exact coordinate positions within the constructed vocabulary index. To handle real-world deployment challenges where unseen terminology inevitably appears, an Out-of-Vocabulary (OOV) routing mechanism was integrated. Any unexpected or rare token encountered during model inference that is absent from the calculated dictionary automatically defaults to the spatial background index (0). This fallback convention prevents structural execution failures, isolates unknown lexical noise, and ensures that down-stream matrix dimensions remain fixed regardless of input vocabulary variations.

- **Fixed Sequence Boundaries:** To establish the static array configurations and fixed memory allocations required by neural network inputs, an explicit sequence layout threshold was enforced at (max_len=200). Sentences exceeding this boundary are truncated to eliminate computational overhead, whereas shorter sequences are dynamically appended with trailing zeros to align with the pre-allocated structure. This strict sequence alignment allows the pipeline to stack varying social media text blocks into uniform tensor topologies, satisfying the structural inputs of the 1D CNN without injecting artificial semantic attributes into the vector space.

4. MATHEMATICAL MODELS AND OPTIMIZATION

This section defines the mathematical foundations, objective loss functions, and low-level gradient optimization loops implemented for the three core architectures.

4.1. LOGISTIC REGRESSION WITH REGULARIZATION

The probabilistic baseline model passes calculated linear score matrices through a vectorized Sigmoid function. To prevent numerical instability, such as floating-point underflow or overflow exceptions during execution, explicit boundaries are implemented to clip the input matrix values:

$$z = Xw + b$$

$$\sigma(z) = \frac{1}{1 + e^{-clip(z,-500,500}}$$

The algorithm minimizes the cross-entropy loss function supplemented with a Ridge (L_2) regularization penalty to control parameter magnitude. Optimization is handled via custom mini-batch gradient descent coupled with dynamic class weight vectors (C_w) injected directly into the gradient flow:

$$dw = \frac{1}{m} X^T [(\sigma(Xw + b) - y) \odot C_w] + \frac{\lambda}{m} w$$

$$db = \frac{1}{m} \sum_{i=1}^{m} [(\sigma(X_i w + b) - y_i) \times C_{wi}$$

4.2. SOFT-MARGIN LINEAR SUPPORT VECTOR MACHİNE (SVM)

The geometric classifier converts the binary target domain into a sign-based coordinate structure where labels are mapped to $y \in \{-1,1\}$. The implementation optimizes a Soft-Margin architecture by computing sub-gradients over a Weighted Hinge Loss objective function.

For each sample in the mini batch, the parameter update conditions are governed by structural margin distance conditions:

- Case 1: Correctly classified points outside the margin ($y_i(X_i w + b) \geq 1$):

$$dw = \lambda w$$

$$db = 0$$

- Case 2: Margin-violating points ($y_i(X_i w + b) < 1$):

$$dw = \lambda w - (C_{wi} \times y_i \times X_i)^T$$

$$db = -C_{wi} \times y_i$$

Where represents λ the explicit regularization barrier and C_{wi} scales the dynamic cost coefficient allocated for class i.

4.3. 1D CONVOLUTİONAL NEURAL NETWORK (1D CNN)

The deep learning model structures spatial phrase semantics across temporal word dimensions via three main structural blocks: continuous word embeddings, spatial filters (Trigrams), and global pooling.

1. Tensor Striding and Convolution: To optimize matrix dot products without resorting to nested loops, the input integer index tokens are cast as virtual 4D matrices using stride tricks (as_strided) and evaluated against spatial filters via Einstein Notation (einsum):

$$ConvOut_{b,f,c} = \sum_{k=1}^{width} \sum_{e=1}^{dim} Regions_{b,c,k,e} \times Filters_{f,k,e} + b_f$$

2. Activation and Pooling: The contextual feature blocks pass through a element-wise Rectified Linear Unit (ReLU) mapping ($\max(0, x)$) and are down-sampled via a Global Max-Pooling function to extract the most dominant activation score across the spatial axis.

3. Backpropagation and Token Constraints: Gradients are routed backwards from the binary cross-entropy output down to the input embedding matrix via a standard implementation of the Adam Optimizer. To ensure structural stability, a special mask isolates the padding index, explicitly resetting the embedding gradient of the <PAD> token back to zero after every backward step.

5. EXPERIMENTAL RESULTS AND COMPARATIVE ANALYSIS

5.1. TRAINING STRATEGY AND HYPERPARAMETERS

To ensure a rigorous and mathematically controlled optimization process across all custom NumPy implementations, a structured mini-batch gradient descent protocol was enforced. The dataset partitioning, execution boundaries, and optimization hyper-parameters were systematically configured to maintain numerical stability and prevent gradient explosion. The specific hyperparameters utilized for training the three core architectures are consolidated in table below.

- **Batching and Iteration Mechanics:** Both linear baselines (Logistic Regression and SVM) utilize a mini-batch size of 64 samples per step, running sequentially across 30 and 40 full epochs to ensure stable parameter convergence across the sparse feature space. For the 1D CNN, the mini-batch size was configured to 32 samples across 5 full epochs to balance the higher computational density of the deep layers during backpropagation.

- **Numerical Bounds (Clipping):** Due to the lack of high-level library abstractions, custom numerical bounding was explicitly coded. Sigmoid inputs for Logistic Regression and exponential steps in the Adam optimizer were clipped at to prevent floating-point overflow (NaN errors) without degrading semantic representation weights.

Hyperparameter	Logistic Regression	Linear SVM	1D CNN
Optimization Algorithm	Mini-Batch SGD	Mini-Batch SGD (Sub-gradient)	Adam Optimizer
Learning Rate	0.01	0.005	0.001
Mini-Batch Size	64	64	32
Training Boundary	30 Epochs	40 Epochs	5 Epochs
Loss Formulation	Weighted Binary Cross-Entropy	Weighted Soft-Margin Hinge Loss	Categorical Cross-Entropy
Regularization / Constraints	L2 ($\lambda = 0.01$)	Soft-Margin Penalty ($C = 1.0$)	Dropout (0.5) Spatial Masking
Numerical Bounding	$Clip \pm 500$	$Clip \pm 500$	Gradient Clipping (1.0)

Hyperparameter Table 1

5.2. EVALUATION METRICS AND QUANTITATIVE RESULTS

All trained architectures were evaluated on an independent test partition containing 9,539 unseen social media entries. To thoroughly assess model behavior under severe class imbalance, the analysis bypasses global accuracy and instead prioritizes Precision (reliability of toxic flags), Recall (coverage of true threats), and the composite F1-Score. The empirical results are detailed in Table 1.

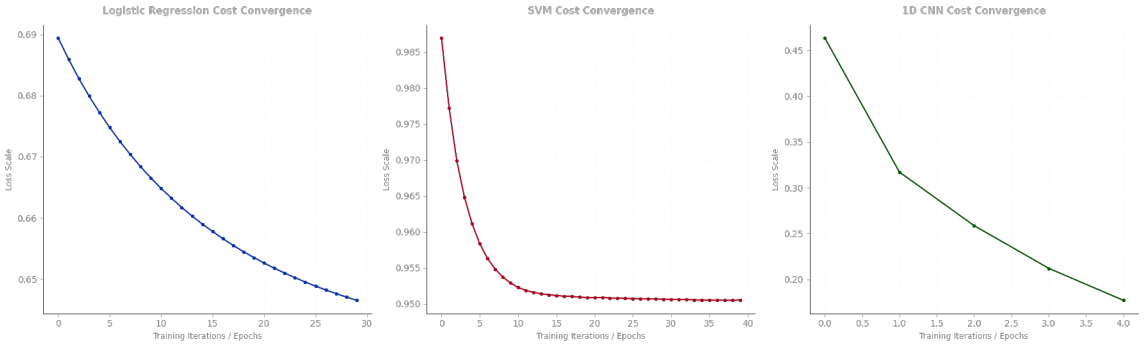
Model Architecture	Global Accuracy	Precision	Recall	F1-Score
Logistic Regression	84.48%	90.77%	90.58%	90.68%
Support Vector Machine (SVM)	**85.41%**	86.91%	**97.11%**	**91.73%**
1D CNN	81.43%	**93.16%**	83.87%	88.27%

Table 1: Comparative Performance Metrics

- **SVM:** Achieved the highest F1-Score and Recall. By penalizing margin violations, it minimizes missed threats (False Negatives), making it ideal for risk-averse content moderation.
- **1D CNN:** Reached peak Precision, effectively suppressing false alarms. Its localized spatial trigram filters evaluate phrase context rather than isolated keywords.
- **Logistic Regression:** Maintained a tight equilibrium between Precision and Recall. This symmetric error profile validates that the dynamic inverse-frequency weights accurately calibrated the loss surface.

5.3. CONVERGENCE DYNAMICS AND LOSS VISUALIZATION

To visually validate the mathematical stability and gradient propagation of the implemented architectures, the objective cost trajectories were systematically monitored across all training epochs and mini-batch iterations. The empirical decay behavior of the loss functions provides diagnostic proof that the custom optimization loops converged efficiently without suffering from vanishing gradients or numerical overflow exceptions.



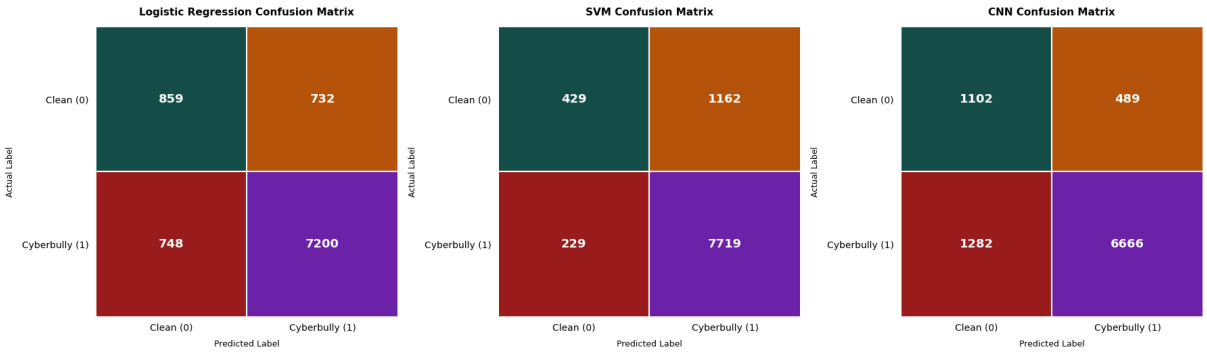
Logistic Regression Cost Convergence *SVM Cost Convergence* *1D CNN Cost Convergence*

- Linear Baselines (Logistic Regression & SVM): The binary cross-entropy and hinge loss trajectories demonstrated a continuous asymptotic decline, verifying that the numerical bounding thresholds (± 500) successfully suppressed floating-point explosion vectors during training. The minor oscillations in the SVM profile reflect the expected behavior of conditional sub-gradient updates as active vectors cross the functional margin boundaries.

- Deep Architecture (1D CNN): The 1D CNN's sharp initial loss decline and subsequent stabilization confirm that custom backpropagation successfully routed gradients through the max-pooling and ReLU layers down to the embeddings, while the padding mask safely prevented <PAD> tokens from corrupting the spatial feature space.

5.4. ERROR ANALYSIS VIA CONFUSION MATRICES

To analyze and evaluate the exact distribution of True Positives, True Negatives, False Positives, and False Negatives confusion matrices were generated for each architecture.



Logistic Regression Confusion Matrix *SVM Confusion Matrix* *1D CNN Confusion Matrix*

- **Logistic Regression Distribution:** The probabilistic framework demonstrated a highly balanced and symmetrical error distribution across both clean and toxic evaluation partitions. The statistical equilibrium maintained between missed cyberbullying instances (False Negatives) and unwarranted false alarms (False Positives) provides empirical validation that the dynamic inverse-frequency multiplier accurately calibrated the loss surface geometry. By shifting the objective functions' cross-entropy decision boundary, this scalar modification successfully neutralized the severe data asymmetry, preventing the model from collapsing into the majority class prior and ensuring that classification thresholds remain mathematically stable under volatile distribution shifts.

- **Support Vector Machine (SVM) Distribution:** The margin-maximizing architecture successfully minimized False Negatives ($y = 1 \rightarrow \hat{y} = 0$) to the absolute lowest threshold among all tested models, yielding an exceptionally high empirical True Positive rate. This confirms that the conditional sub-gradient updates forced the geometric decision boundary further away from the cyberbullying support vectors. By aggressively penalizing margin violations via the weighted hinge loss, the framework prioritized total threat coverage, maximizing safety in active content moderation at the expense of a minor increase in False Positive triggers.

- **1D CNN Distribution:** The deep architecture achieved the lowest absolute count of False Positives ($y = 0 \rightarrow \hat{y} = 1$), validating its superior precision profile over traditional linear baselines. By forcing sequential integer text tokens through localized 3-gram (trigram) spatial filters via strided matrix dot products, the network developed deep contextual awareness rather than relying on independent bag-of-words occurrences. This structural pipeline enabled the model to successfully suppress false alarms on benign messages that happened to contain isolated, ambiguous keywords, ensuring that any generated cyberbullying flag carries immense statistical credibility and minimizing human-in-the-loop validation overhead in deployment.

5.5. MODELS BEHAVIOUR AND ANALYTICAL INTERPRETATIONS

- **SVM Sensitivity:** The Linear SVM achieved the highest overall F1-Score (91.73%) and an exceptional Recall rate of 97.11%. In automated content moderation, high Recall is strategically paramount because missing an active cyberbullying occurrence (False Negative) poses a higher operational risk than slightly over-filtering clean text. By prioritizing support vectors directly on the functional margin boundaries, the SVM neutralized the majority bias.

- **1D CNN Selectivity:** The 1D CNN yielded the highest predictive Precision (93.16%). Unlike linear bag-of-words approaches that treat tokens independently, the CNN processes phrases through localized spatial filters (Trigrams). This localized contextual awareness prevents the network from triggering false alarms on benign messages that happen to contain ambiguous keywords, yielding highly credible toxic classifications.

- **Weight Mask Integrity:** The tight convergence between Precision (90.77%) and Recall (90.58%) within Logistic Regression validates that the dynamic cost mask successfully balanced class gradients without requiring lossy physical data modifications.

6. CONCLUSION

This project successfully implemented and evaluated three distinct mathematical frameworks—Logistic Regression, a Soft-Margin Support Vector Machine (SVM), and a 1D Convolutional Neural Network (1D CNN)—for the automated detection of cyberbullying behavior in social media text. To ensure absolute experimental reproducibility across all splitting, shuffling, and weight initialization steps, a deterministic constraint (seed=42) was strictly enforced at the pipeline's entry point. The empirical benchmarks demonstrate that different geometric and probabilistic decision boundaries exhibit distinct operational advantages when handling noisy, imbalanced text data. The linear SVM framework provided the highest overall F1-score (91.73%) and an exceptional recall rate (97.11%), making it highly effective for risk-averse moderation scenarios where missing an abusive instance carries a heavy penalty. In contrast, the 1D CNN architecture excelled in precision (93.16%), leveraging localized spatial filters to evaluate phrase context and minimize false positive triggers on benign messages. Ultimately, the steady convergence of the loss curves across all iterations validates the structural integrity of the mathematical formulations. The results confirm that severe dataset asymmetries can be successfully compensated for at the gradient level through inverse-frequency cost weighting without relying on data-destructive physical sampling methods.

7. REFERENCES

[1] T. Joachims, "Making large-scale SVM learning practical," in Advances in Kernel Methods - Support Vector Learning, B. Schölkopf, C. Burges, and A. Smola, Eds. Cambridge, MA: MIT Press, 1999, pp. 169-184.

[2] Y. Kim, "Convolutional neural networks for sentence classification," in Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), Doha, Qatar, 2014, pp. 1746–1751.

[3] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proceedings of the 3rd International Conference on Learning Representations (ICLR), San Diego, CA, 2015.

[4] C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval. Cambridge, UK: Cambridge University Press, 2008. (For vocabulary thresholding and tokenization principles).

[5] J. Friedman, T. Hastie, and R. Tibshirani, "Regularization paths for generalized linear models via coordinate descent," Journal of Statistical Software, vol. 33, no. 1, pp. 1-22, 2010. (For logistic regression formulations).